



The ministry of silly builds

a Bazel 9 derivation builder

Aleksander Gondek | Artur Stachecki



Aleksander Gondek

Alex is well-known for spearheading migrations to Bazel build system at scale. He enjoys solving complex challenges in a manner that results in simple solutions. Rust, Nix and Bazel enthusiast, he attempts to bring reproducibility and correctness to any software project he works on. His mantra? 'There has to be a better way!'.

✉ gondekaleksander@protonmail.com

🐙 <https://github.com/AleksanderGondek>



Artur Stachecki

Artur has background in computer science and engineering, focuses on correctness, reliability, and reusability of software systems. Leading large-scale migrations, he excels in creating advanced build systems and ensuring build correctness. Artur loves engineering innovative solutions using variability modeling.

✉ contact@r2r.sh

🐙 <https://github.com/r2r-dev>

```
[nixcon@krakow:~]$ ./the-ministry-of-silly-builds//
```



Introduction

```
[nixcon@krakow:~]$ ./the-ministry-of-silly-builds/introduction/
```



```
[nixcon@krakow:~]$ ./the-ministry-of-silly-builds/introduction/why-running-bazel-under-nix-is-non-trivial?
```

Why running Bazel under nix is non-trivial?

```
[nixcon@krakow:~]$. /the-ministry-of-silly-builds/introduction/why-running-bazel-under-nix-is-non-trivial?
```

- **Bazel** model of downloading external dependencies is skewed towards laziness (some things will be only downloaded when they are about to be used)

```
[nixcon@krakow:~]$ ./the-ministry-of-silly-builds/introduction/why-running-bazel-under-nix-is-non-trivial?
```

- **Bazel** ruleset authors have significant freedom in shaping how said external dependencies are obtained (it is possible to run arbitrary process and capture its outputs (*rctx.execute*))

```
[nixcon@krakow:~]$ ./the-ministry-of-silly-builds/introduction/why-running-bazel-under-nix-is-non-trivial?
```

- **Bazel** and its rulesets frequently do not shy away from hardcoding certain assumptions about Linux runtime

```
[nixcon@krakow:~]$. /the-ministry-of-silly-builds/introduction/state-of-art
```

State of art

```
[nixcon@krakow:~]$ ./the-ministry-of-silly-builds/introduction/state-of-art
```

buildBazelPackage / bazelPackage:

- focused on using **Bazel** to deliver binaries into */nix/store* ecosystem
- **Bazel** *fetch* and/or *vendor* outputs stored in */nix/store*
- **Bazel** and its dependencies are patched to reference */nix/store*

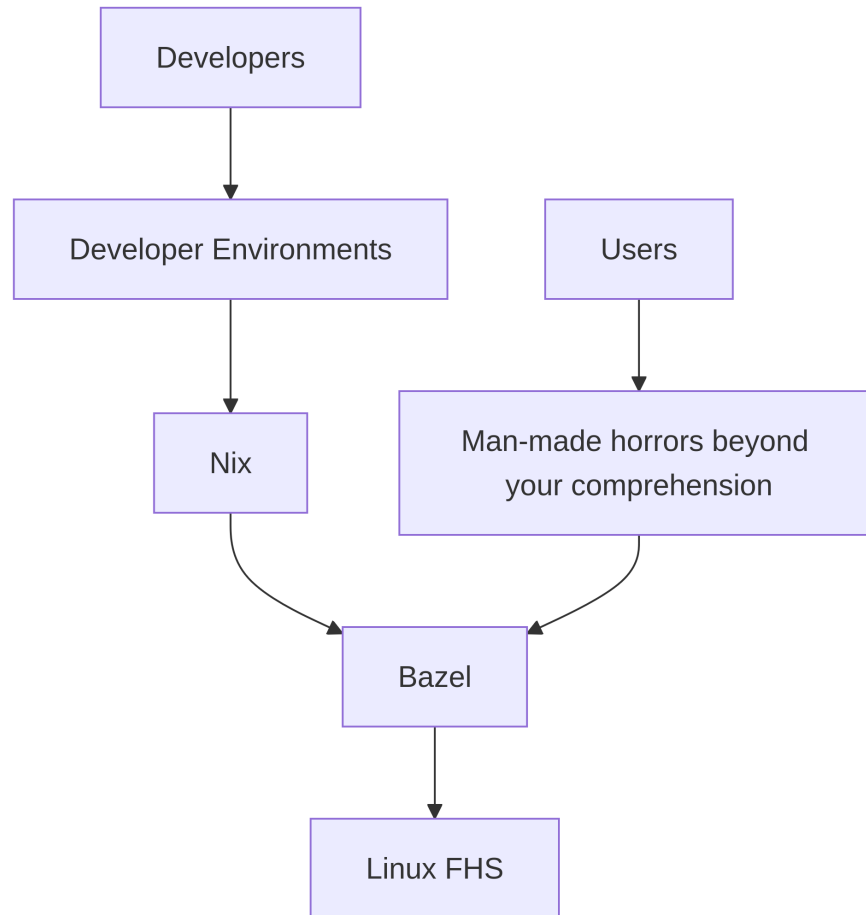
```
[nixcon@krakow:~]$ ./the-ministry-of-silly-builds/introduction/state-of-art
```



```
[nixcon@krakow:~]$. /the-ministry-of-silly-builds/introduction/our-usecase
```

Our usecase

[nixcon@krakow:~]\$./the-ministry-of-silly-builds/introduction/our-usecase



```
[nixcon@krakow:~]$. /the-ministry-of-silly-builds/introduction/our-usecase
```

- In other words, Nix must not be visible in neither **Bazel** build system, nor output binaries.
- Also: consider */nix/store* capacity if each **Bazel** has separate vendored derivation.

```
[nixcon@krakow:~]$. /the-ministry-of-silly-builds/introduction/goals
```

- Goal 1: **Bazel** and its build inputs remain untouched by *nix* patching
- Goal 2: **Bazel** build outputs remain unaware of *nix*
- Goal 3: **Nix** derivations are granular enough to allow for artifacts sharing

[nixcon@krakow:~]\$./the-ministry-of-silly-builds/introduction/side-note

- With Bazel 9+ usage of *bzlmod* is mandatory. It was designed to bring modern dependency resolution experience to Bazel.
- It has *MODULE.bazel.lock* files!
- Surely the lockfile contains all the outputs that is registered by *module_extensions* that can execute arbitrary code
- ...

Demo !

```
[nixcon@krakow:~]$ ./the-ministry-of-silly-builds/demo!/questions?
```



Questions?